ForkSysInfo 系統設計與執行驗證報告

C 語言網路程式作業 2026 年 11 月 13 日

本專案以 C 語言實作 TCP 用戶端與伺服器。伺服器透過 `fork()` 為每條連線建立獨立子行程，不使用執行緒。雙方動態連結共同的 `libnetsys.so`，並提供編譯時及執行時兩層 debug 控制。系統資訊查詢是主要功能，另外提供 PING、ECHO、SLEEP 與 QUIT，以利驗證協定、併發和連線生命週期。

本次執行平台為 Windows 本機上由 QEMU 軟體模擬的 Linux 環境；Docker 與原生 Linux 是可選的重現方式。SYSINFO 顯示 Linux 環境所見的核心、主機名稱及記憶體等資訊，不代表 Windows 工作管理員的數值。

2026 年 11 月 13 日 11 時 50 分 56 秒至 11 時 51 分 05 秒（UTC+8）的 11 項整合驗證全部通過。環境紀錄為 Linux `6.12.51-0-virt`、GCC 14.2.0、GNU Make 4.4.1、Python 3.12.16 與 tcpdump 4.99.5。原始 log 採 UTC 時間，故對應 03:50:56 至 03:51:05；兩者為同一次實測。

1 系統架構

父行程建立 TCP listening socket，執行 `bind()`、`listen()`，接著在接收迴圈中 `accept()`。每成功接受一條連線，父行程呼叫 `fork()`。子行程關閉 listening socket，處理該連線的請求並結束；父行程關閉已交給子行程的 connected socket，立即繼續接受其他連線。雙方分別關閉不再使用的描述符，避免其他行程仍持有 socket 而延後 EOF。

```
C client 1 ── TCP ── child process 1
C client 2 ── TCP ── child process 2
    ...              ...
C client 12 ─ TCP ── child process 12
                  ↑ fork
            parent listening process
```

client → libnetsys.so ← server

父行程管理子行程的數量與回收，並處理中斷或終止訊號。預設的用戶端數量上限為 64，超過上限時拒絕新增工作，避免無限制建立行程。預設綁定 `127.0.0.1`，讓展示與封包擷取都在同一台電腦完成。

伺服器子行程支援在同一條連線處理多個 frame，直到收到 QUIT、EOF 或錯誤。命令列 client 每次執行只送一個請求，收到一個回應後關閉連線。兩者的生命週期設計不同，但使用同一個 framing 協定。

`fork()` 讓各連線有獨立位址空間，單一子行程的連線錯誤不會直接結束父行程。代價是每條連線需要額外的行程資源，因此必須限制數量、設定逾時，並回收已結束的子行程。[1][2]

2 應用層協定

TCP 提供有序的位元組串流，不會保留應用程式每次 `send()` 的邊界。因此一次傳送可能由多次接收取得，多次傳送也可能由一次接收取得。[3] 本專案採用長度前綴，明確界定每個 frame。

欄位	大小	意義
payload length	4 bytes	unsigned 32-bit，network byte order
payload	1 至 4096 bytes	文字命令或回應，不包含結尾 NUL

例如 PING 的應用層資料是 00 00 00 04 50 49 4e 47。前 4 bytes 的數值為 4，後 4 bytes 是 ASCII 命令。程式不假設這 8 bytes 一定出現在同一個 TCP 封包中。

命令	功能	回應內容
SYSINFO	查詢伺服器所見的系統資訊	uname、hostname、pid、uptime、RAM
PING	確認應用程式可回應	OK PONG 與子行程 PID
ECHO text	回傳文字	OK 與原始文字
SLEEP ms	模擬較長的工作	完成等待後回傳耗時設定與 PID
QUIT	結束連線	結束回應

成功回應以 OK 開頭；可回覆的命令錯誤以 ERR 開頭。錯誤的長度、提早關閉或無法繼續解碼的 frame 會結束該連線。長度在讀取 payload 前先檢查，不依照未驗證的長度配置任意大小記憶體。

文字 payload 中的內嵌 NUL 也會被拒絕，避免 C 字串在中間結束，導致實際收到的 frame 與命令解析內容不同。SLEEP 只接受 0 到 10000 毫秒的整數。這些限制在比較模式中仍保留。

3 共用動態函式庫與原始碼

include/ 宣告共同介面，src/ 提供伺服器、用戶端及函式庫實作。共同的封包處理、數值檢查、時間與 log 功能集中在 libnetsys.so，避免雙方各維護一份相同程式。所有完整原始檔隨專案繳交，並可由 Makefile 重建。

原始檔與函式	主要責任
include/netsys.h	宣告協定常數、共享 API 與 NET_DEBUG 巨集
src/netsys.c 的 net_recv_frame()、net_send_frame()	接收與傳送長度前綴 frame，處理完整 I/O 與期限
src/netsys.c 的 net_connect()、net_listen()	建立 IPv4 連線及 listening socket
src/netsys.c 的 net_parse_long()、net_log()	檢查數值範圍與產生帶有行程資訊的 log
src/server.c 的 system_info()、handle_command()	取得 Linux 資訊，執行各項命令
src/server.c 的 serve_client()	子行程的持續收送迴圈與連線結束處理
src/server.c 的 reap_children()	以 waitpid(-1, ..., WNOHANG) 回收完成的子行程
src/server.c 的 main()	解析選項、accept、fork、限制連線數及關閉清理
src/client.c 的 main()	解析命令列、連線、發出單次請求並顯示回應

動態函式庫以位置獨立程式碼建置，再以 shared library 方式連結。client 與 server 的 ELF 動態資訊會列出 libnetsys.so 相依性；ldd 顯示執行時實際載入的路徑。執行檔的 RUNPATH 使用 $ORIGIN，讓它們在自

己的目錄中找到函式庫。這使 debug 與 release 可同時存在，不需要修改全域函式庫搜尋路徑。

Makefile 將 debug 與 release 的目標分別放入 `build/debug/` 和 `build/release/`。`make DEBUG=1` 建立前者；`make DEBUG=0` 建立後者。這個設計避免切換編譯旗標時誤用先前的 `.o` 檔案。[5]

4 兩層 debug 控制

第一層由 Makefile 傳入編譯期旗標。關閉時，debug 巨集不產生輸出呼叫；第二層由執行期 `--debug` 決定是否輸出仍存在於程式中的 debug 訊息。編譯期關閉後，執行期旗標無法把已移除的程式碼恢復。

編譯設定	執行旗標	實測 debug 行數
`DEBUG=1`	無 `--debug`	0
`DEBUG=1`	有 `--debug`	8
`DEBUG=0`	無 `--debug`	0
`DEBUG=0`	有 `--debug`	0

一般錯誤、啟動與重要生命週期訊息仍會保留，方便在關閉 debug 的版本診斷連線失敗。

上表統計單次 PING 交易的 client 與 server log。8 行包含連線、frame 傳送與接收、命令處理及 EOF 等 debug 記錄；不是要求所有操作固定產生 8 行。

5 健全性機制

正常模式使用完整接收與傳送迴圈，處理 partial I/O、`EINTR` 和提早斷線。frame 的接收以單調時鐘計算截止時間，避免對方每隔一小段時間只傳一個 byte，就無限延長資源占用。固定長度界限與命令參數檢查則限制不合法輸入。

程式抑制 `SIGPIPE` 導致的突然終止，將對方關閉後的寫入錯誤當作一般錯誤處理。父行程回收子行程，避免 zombie；連線數量有上限；伺服器結束時會清理仍存在的子行程。連線拒絕、埠號占用與系統呼叫失敗均以非零結束碼或明確 log 呈現。

socket 使用 nonblocking I/O 並搭配 `poll()` 等待 readiness。父行程的 `SIGCHLD` handler 只負責打斷等待，不在 handler 裡執行格式化輸出或修改複雜資料；回收工作在一般控制流程中完成。父行程最長每 100 毫秒重新檢查一次，避免完成的子行程長期留存。接收錯誤在可回傳時會送出 `ERR TIMEOUT`、`ERR BAD_LENGTH` 或 `ERR BAD_FRAME` 後關閉。

建立子行程時，程式暫時阻擋 SIGTERM 與 SIGINT。子行程先把終止 handler 改回預設值，再解除阻擋；父行程先記錄新 PID，再解除阻擋。這避免在 fork 前後立刻關閉伺服器時，子行程沿用父行程只設定旗標的 handler，卻仍停在接收迴圈中，造成關閉程序無法結束。

為比較效果，`--unsafe` 僅關閉下表中的三項機制。固定緩衝區與最大長度限制仍然保留，因此這個模式不是任意記憶體破壞範例，也不是正式部署模式。

情境	正常模式	`--unsafe` 比較模式
4-byte header 被分段傳送	迴圈讀滿後正確解析	單次接收取得不足 4 bytes，請求失敗
接受連線後不傳完 frame	到截止時間後結束子行程	子行程繼續等待，占用資源

情境	正常模式	`--unsafe` 比較模式
用戶端完成並離開	父行程呼叫 `waitpid()` 回收	子行程留在 zombie 狀態，直到清理

上述差異必須看實際結果，不能只憑原始碼判斷。測試使用相同的命令、等待時間與觀察方式，分別執行正常模式及比較模式，再保存真實 log 與 `/proc` 行程狀態。[2]

三組實際執行結果如下。PID 屬於本次執行，重跑時會不同。

執行案例	正常模式實測	比較模式實測
PING frame 分成 1、2、5 bytes 傳送	`OK PONG pid=2114`	連線已被關閉，測試端後續寫入出現 `BrokenPipeError`
連線後不傳資料，期限設為 500 ms	1.207 秒後觀察，PID 2118 已被回收，存活子行程 0 個	1.219 秒後仍有 PID 2120，狀態 S，存活子行程 1 個
PING 完成並關閉 client	子行程 PID 2123 已被回收，zombie 0 個	PID 2126 留在 Z 狀態，zombie 1 個

閒置案例是在約 1.2 秒後取樣，不代表逾時設定是 1.2 秒；真正的 deadline 設定為 500 毫秒。分段案例的 1、2、5 bytes 合計包含 4-byte header 與 4-byte PING，其中第一段只有 1 byte，足以暴露單次接收的錯誤假設。測試結束時仍會清理比較模式的行程。

6 同時服務至少十個用戶端的證明

驗證程式同時啟動 12 個 C client，要求伺服器執行 SLEEP 工作。在所有工作尚未完成的重疊期間，擷取父子行程關係、子行程 PID 與 TCP 連線狀態，再檢查 12 個 client 均收到成功回應。

僅有 12 行成功輸出不足以證明同時服務，因為依序執行也能得到相同輸出。本次在父行程 PID 2082 下，同時觀察到 12 個子行程：2085、2087、2089、2091、2093、2094、2097、2098、2101、2103、2104、2106。它們在快照中均為 S 狀態，對應各自仍在執行的 SLEEP 工作。

12 個 C client 都收到 `OK SLEPT 2000`，結束碼均為 0，總耗時為 2.141 秒。若完全依序執行，僅等待工作本身的下限就是 12 × 2 = 24 秒。這個 24 秒是理論下限，並非另外量測出的循序版本時間。主要證據是同時存在的 12 個不同子行程與 TCP 連線，耗時是輔助證據。`ss` 在同一主機會列出每條連線的兩端，不能把 24 個 loopback endpoint 誤算為 24 個 client。

7 封包擷取與分析

測試使用 `tcpdump` 在 `lo` 介面擷取指定測試埠號的流量，將二進位封包保存為 `.pcap`。讀檔命令再輸出時間、來源與目的埠號、TCP flags、序號和 payload。這是同一台 Linux 主機上的 loopback 通訊，符合使用 tcpdump 的條件。[4]

```
tcpdump -i lo -U -w evidence/loopback.pcap 'tcp port 19090'
tcpdump -nn -tttt -r evidence/loopback.pcap
tcpdump -nn -X -r evidence/loopback.pcap
```

分析重點包括三向交握的 SYN、SYN ACK、ACK；應用層 request 與 response；長度前綴與命令文字；以及關閉時的 FIN 或異常時的 RST。TCP segment 與應用層 frame 不是一對一關係，分析 payload 時必須依序號重組同一方向的串流，並依 4-byte 長度解碼。

本次 loopback.pcap 為 3663 bytes，包含 4 條 TCP 連線、40 個封包，其中 8 個含應用資料、8 個帶 SYN flag、8 個帶 FIN flag，沒有 RST。SYN 計數包含 4 個 SYN 與 4 個 SYN ACK，因此代表 4 條連線。4 個請求分別為 SYSINFO、PING、ECHO 與 SLEEP 50；四個請求與四個回應共形成 8 個已解碼的應用層 frame。

以第一條連線為例，client 端為 127.0.0.1:57382，server 端為 127.0.0.1:19090。封包 1 至 3 是三向交握；封包 4 的 TCP payload 為 11 bytes，其中 4 bytes 的 00 00 00 07 表示後面 7 bytes 為 SYSINFO。封包 6 的 payload 為 166 bytes，由 4-byte 長度 00 00 00 a2 與 162-byte 回應組成。伺服器對 client 資料的 ACK 從 847142362 增加至 847142373，正好涵蓋 11 bytes；回應方向也可用相同方式檢查 166 bytes。封包 8、9、10 顯示 FIN ACK、FIN ACK 與最後 ACK 的正常關閉。

SYSINFO 回應記錄主機名稱 localhost、Linux release 6.12.51-0-virt、架構 x86_64、子行程 PID 2150、uptime 210 秒及 RAM 資訊。另一條 PING 連線的請求 header 為 00 00 00 04，回應為 OK PONG pid=2152，與應用程式的長度前綴規格一致。

loopback 的封包時間非常接近，且可能受到核心 segmentation 或 checksum offload 的呈現方式影響。不能僅因擷取工具顯示 checksum 提示，就斷言應用程式資料毀損。原始 pcap 可用 Wireshark 開啟，以 Follow TCP Stream 交叉檢查。

8 執行證據與重現方式

請由專案根目錄執行 bash scripts/run_all.sh。它會建置兩種版本並執行驗證。evidence/ 保存環境資訊、執行 log、結果 JSON、併發行程快照、debug 輸出以及 pcap。隨附的執行截圖是實際原始 log 檢視器的畫面擷取；兩分鐘影片是有旁白的證據講解，並非未剪輯的全程桌面錄影。

本次結果為 11 項整合驗證全部通過。其中連線上限另以 --max-clients 2 驗證，前兩個 child 保留時，第三個 client 收到 ERR SERVER_BUSY。共享函式庫檢查確認 client 與 server 都有 DT_NEEDED libnetsys.so，並使用 ORIGIN 相對執行檔的搜尋路徑。

證據檔案	用途
evidence/results.json	每項驗證的通過狀態、耗時與量測結果
evidence/raw/concurrency.txt	十二個 C client 的執行結果
evidence/raw/concurrency_processes.txt	重疊期間的父子行程與 PID
evidence/raw/concurrency_ss.txt	TCP established 連線快照
evidence/raw/robustness_fragmented.txt	分段 header 的兩種模式比較
evidence/raw/robustness_idle.txt	閒置連線的兩種模式比較
evidence/raw/robustness_zombies.txt	完成後的 zombie 行程比較
evidence/raw/debug_matrix.txt	編譯與執行兩層控制的四種組合
evidence/loopback.pcap	tcpdump 取得的原始封包

證據檔案	用途
`evidence/raw/tcpdump_hex.txt`	封包十六進位與文字內容
`evidence/raw/decoded_frames.txt`	依 TCP 串流解碼的應用層 frame
`evidence/packet_analysis.json`	封包及串流分析的結構化結果

其他異常驗證包括零長度、過大與 UINT32 最大長度、內嵌 NUL、最大合法長度、同連線合併傳入 PING 與 QUIT、對方於 SLEEP 期間以 RST 斷線、埠號占用、連線被拒、錯誤命令或參數，以及較低連線上限的 admission control。每個案例均驗證可觀察的結果，並在需要時確認父行程仍可服務後續請求。

目前的證據應與同一份原始碼及建置結果一起檢視。若修改程式或在另一台電腦執行，請重新產生證據，不沿用舊 PID、舊封包或舊測試結果。

9 現場展示程序與限制

在自己的電腦準備好可用的 Linux 環境，先執行一次完整驗證。Windows 可使用隨附的 QEMU 方式；若 Docker 的 Linux backend 已可啟動，也可自行建置映像檔。上課依序展示一般系統資訊查詢、12 個用戶端的同時服務證據、三組健全性差異、debug 四種組合及封包分析。重新執行測試可以直接產生自己的新證據。

Windows 隨附入口為根目錄的 `START_DEMO.cmd`，用來啟動可攜式 Linux、更新主機端原始碼並執行完整驗證；`LINUX_SHELL.cmd` 用來進入可手動操作的 Linux 終端機，並自動切換至 `/root/ForkSysInfo`。可開兩個終端機，分別執行 server 與 client。新證據會存到 `runs/` 下的時間戳記資料夾，完成後使用 `STOP_LINUX.cmd` 正常關閉 Linux。請保留整個專案及 `portable-linux/` 的相對位置。

可攜式磁碟安裝後採用 Linux 6.12.117 核心，並再度通過全部 11 項測試。第二次結果保存在 `evidence-recheck/`，與本報告前述 Linux 6.12.51 的第一次封包及 PID 分開保存。`media/06_live_console.png` 是第二次環境直接執行 C 程式的虛擬機終端畫面，與原始 log 檢視器截圖一併提供。

本專案未加入 TLS 或身分驗證；預設只在 loopback 使用。每連線一個行程的設計適合本作業的十個以上連線示範，但不宣稱可無限制擴充。容器的資源資訊由其 Linux 執行環境提供，與實體 Windows 主機的視角可能不同。

10 參考資料

1. Linux man-pages，fork(2)：https://man7.org/linux/man-pages/man2/fork.2.html

2. Linux man-pages，wait(2) 與 waitpid(2)：https://man7.org/linux/man-pages/man2/waitpid.2.html

3. Linux man-pages，tcp(7)：https://man7.org/linux/man-pages/man7/tcp.7.html

4. The Tcpdump Group，tcpdump manual：https://www.tcpdump.org/manpages/tcpdump.1.html

5. GNU，GNU make manual：https://www.gnu.org/software/make/manual/make.html

執行結果畫面

以下截圖由原始執行紀錄檢視器擷取。內容來自本機 Linux 實測的 stdout、行程快照與封包分析；可與 evidence 目錄中的原始檔逐項核對。

ForkSysInfo 實測總覽



系統資訊與基本功能

系統資訊與基本功能

原始執行紀錄檢視器 · 01_system_info

來源 evidence/raw/normal_and_exceptional.txt · C client 的真實 stdout

```
$ client SYSINFO
OK SYSINFO
hostname=localhost
sysname=Linux
release=6.12.51-0-virt
machine=x86_64
pid=2062
uptime_seconds=203
ram_total_bytes=1546440704
ram_free_bytes=1111281664

$ client PING
OK PONG pid=2064

$ client ECHO hello shared library
OK hello shared library

$ client SLEEP 20
OK SLEPT 20 pid=2068
```

畫面呈現 Linux 程式的實際保存輸出，可與 JSON、原始 log 及 PCAP 逐項核對。

12 個用戶端同時執行

12 個用戶端同時執行

原始執行紀錄檢視器 · 02_concurrency

同一快照 12 個 worker · parent PID 2082 · 實測 2.141 秒 · 逐一等待的理論下限 24.0 秒

/proc 子行程快照

PID	PPID	狀態	名稱
2085	2082	S	server
2087	2082	S	server
2089	2082	S	server
2091	2082	S	server
2093	2082	S	server
2094	2082	S	server
2097	2082	S	server
2098	2082	S	server
2101	2082	S	server
2103	2082	S	server
2104	2082	S	server
2106	2082	S	server

12 個 C client 的回應

```
client 01: OK SLEPT 2000 pid=2085
client 02: OK SLEPT 2000 pid=2087
client 03: OK SLEPT 2000 pid=2089
client 04: OK SLEPT 2000 pid=2091
client 05: OK SLEPT 2000 pid=2093
client 06: OK SLEPT 2000 pid=2094
client 07: OK SLEPT 2000 pid=2097
client 08: OK SLEPT 2000 pid=2098
client 09: OK SLEPT 2000 pid=2101
client 10: OK SLEPT 2000 pid=2103
client 11: OK SLEPT 2000 pid=2104
client 12: OK SLEPT 2000 pid=2106
```

原始證據 concurrency.txt、concurrency_processes.txt、concurrency_ss.txt

畫面呈現 Linux 程式的實際保存輸出，可與 JSON、原始 log 及 PCAP 逐項核對。

三組健全性機制的實測比較

三組健全性機制的實測比較

原始執行紀錄檢視器 · 03_robustness

相同測試輸入，分別使用正常模式與 --unsafe。以下欄位直接讀取 results.json。

```
1. Fragmented header: send 1 byte, then 2, then 5
robust: OK PONG pid=2114
unsafe: BrokenPipeError: [Errno 32] Broken pipe

2. Idle connection: deadline 500 ms; inspect after ~1.2 s
robust: active_children=0
unsafe: active_children=1

3. Completed client: parent must reap worker
robust: zombies=0; children=[]
unsafe: zombies=1; children=[{'pid': 2126, 'ppid': 2124, 'state': 'Z', 'command': 'server'}]
```

原始紀錄 robustness_fragmented.txt、robustness_idle.txt、robustness_zombies.txt

畫面呈現 Linux 程式的實際保存輸出，可與 JSON、原始 log 及 PCAP 逐項核對。

雙層 debug 與動態函式庫

雙層 debug 與動態函式庫

原始執行紀錄檢視器 · 04_debug_library

四組實際輸出計數 · ldd / readelf 連結證據

```
DEBUG=1  --debug=False  DEBUG lines=0
DEBUG=1  --debug=True   DEBUG lines=8
DEBUG=0  --debug=False  DEBUG lines=0
DEBUG=0  --debug=True   DEBUG lines=0
```

兩側載入共用 .so

```
$ ldd build/debug/server
        libnetsys.so => /root/ForkSysInfo/build/debug/libnetsys.so (0x7f5d1dac0000)
$ readelf -d build/debug/server
 0x0000000000000001 (NEEDED)             Shared library: [libnetsys.so]
 0x000000000000001d (RUNPATH)            Library runpath: [$ORIGIN]
$ ldd build/debug/client
        libnetsys.so => /root/ForkSysInfo/build/debug/libnetsys.so (0x7f9db3499000)
$ readelf -d build/debug/client
 0x0000000000000001 (NEEDED)             Shared library: [libnetsys.so]
 0x000000000000001d (RUNPATH)            Library runpath: [$ORIGIN]
$ nm -D --defined-only build/debug/libnetsys.so
```

完整原始紀錄 debug_matrix.txt、shared_library.txt

畫面呈現 Linux 程式的實際保存輸出，可與 JSON、原始 log 及 PCAP 逐項核對。

tcpdump loopback 封包擷取

tcpdump loopback 封包擷取

原始執行紀錄檢視器 · 05_packets

真實 PCAP：40 packets · 4 TCP connections · 8 data segments · 0 RST

```
$ tcpdump -nn -tttt -r evidence/loopback.pcap
2026-11-13 03:51:03.889534 IP 127.0.0.1.57382 > 127.0.0.1.19090: Flags [S], seq 847142361, win 65495, options [mss 65495,sackOK,TS
val 41803953 ecr 0,nop,wscale 7], length 0
2026-11-13 03:51:03.889804 IP 127.0.0.1.19090 > 127.0.0.1.57382: Flags [S.], seq 3018948516, ack 847142362, win 65483, options [mss
65495,sackOK,TS val 41803956 ecr 41803953,nop,wscale 7], length 0
2026-11-13 03:51:03.890128 IP 127.0.0.1.57382 > 127.0.0.1.19090: Flags [.], ack 1, win 512, options [nop,nop,TS val 41803956 ecr
41803956], length 0
2026-11-13 03:51:03.893976 IP 127.0.0.1.57382 > 127.0.0.1.19090: Flags [P.], seq 1:12, ack 1, win 512, options [nop,nop,TS val
41803960 ecr 41803956], length 11
2026-11-13 03:51:03.894082 IP 127.0.0.1.19090 > 127.0.0.1.57382: Flags [.], ack 12, win 512, options [nop,nop,TS val 41803960 ecr
41803960], length 0
2026-11-13 03:51:03.895959 IP 127.0.0.1.19090 > 127.0.0.1.57382: Flags [P.], seq 1:167, ack 12, win 512, options [nop,nop,TS val
41803962 ecr 41803960], length 166
```

解碼後的長度前綴與命令 節錄

```
TCP ephemeral port 57382 request
length=7 header=00 00 00 07
SYSINFO

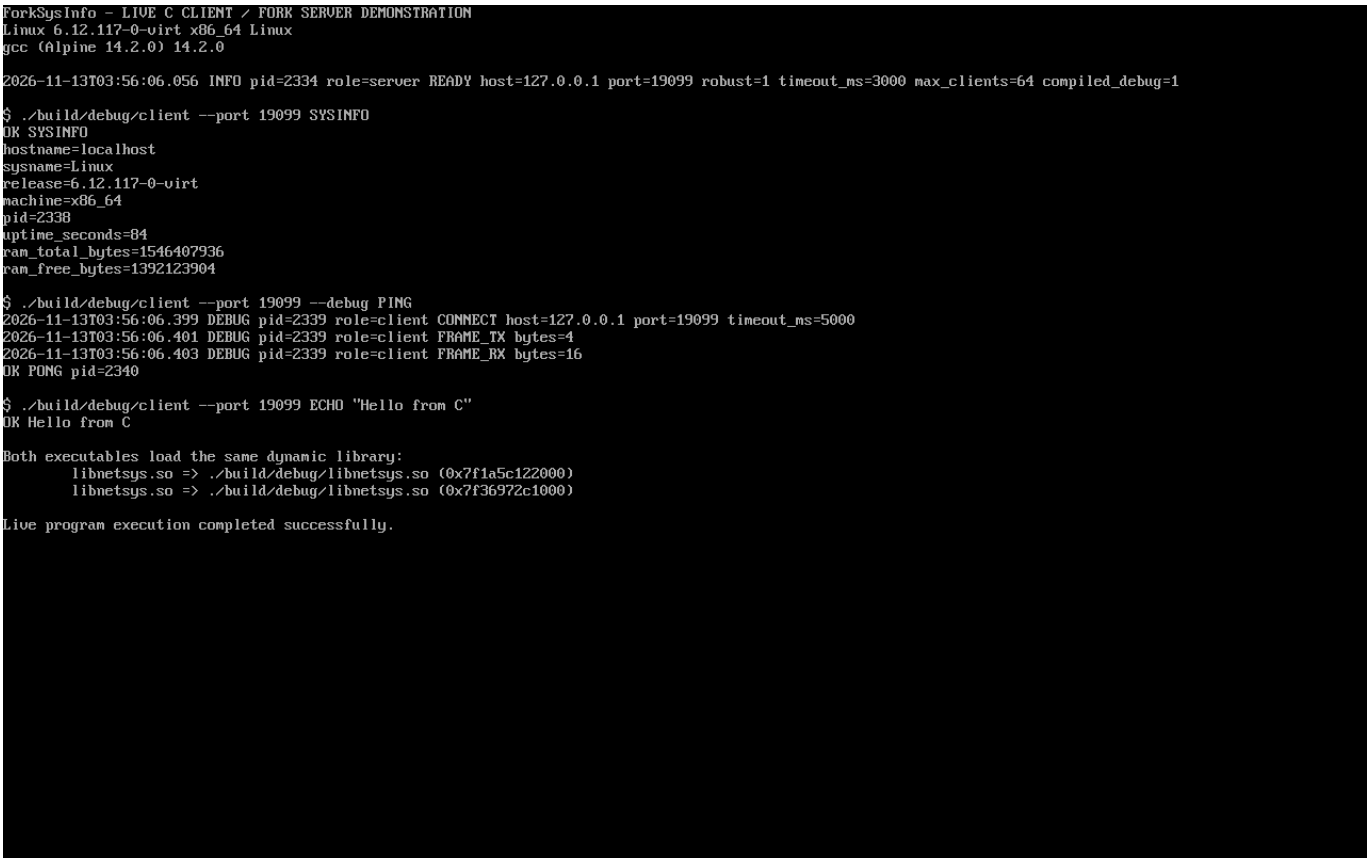
TCP ephemeral port 57382 response
length=162 header=00 00 00 a2
OK SYSINFO
hostname=localhost
sysname=Linux
release=6.12.51-0-virt
machine=x86_64
pid=2150
uptime_seconds=210
ram_total_bytes=1546440704
ram_free_bytes=1101258752
```

檔案 evidence/loopback.pcap · packet_analysis.json · tcpdump_hex.txt

畫面呈現 Linux 程式的實際保存輸出，可與 JSON、原始 log 及 PCAP 逐項核對。

Linux 終端機直接執行畫面

此圖為安裝後 Linux 6.12.117 環境的虛擬機畫面擷取。scripts/console_demo.sh 當場啟動 C server 及 client，直接輸出 SYSINFO、debug PING、ECHO 和 ldd；屬於第二次環境驗證，與前述 Linux 6.12.51 的第一次封包紀錄分開。



完整 C 原始碼與 Makefile

以下內容與隨附的可編輯原始檔相同。測試程式與啟動腳本另完整收錄於專案目錄。

include/netsys.h

```
#ifndef NETSYS_H
#define NETSYS_H

#include <stddef.h>
#include <stdint.h>

#ifndef ENABLE_DEBUG
#define ENABLE_DEBUG 0
#endif

#define NET_MAX_FRAME 4096U
#define NET_DEFAULT_PORT 9090

void net_set_log(const char *role, int debug_enabled);
void net_log(const char *level, const char *format, ...)
    __attribute__((format(printf, 2, 3)));
int net_debug_enabled(void);
#if ENABLE_DEBUG
#define NET_DEBUG(...) do { if (net_debug_enabled()) net_log("DEBUG", __VA_ARGS__); } while (0)
#else
#define NET_DEBUG(...) do { } while (0)
#endif

int64_t net_now_ms(void);
int net_parse_long(const char *text, long minimum, long maximum, long *value);
int net_nonblocking(int fd);
int net_connect(const char *host, int port, int timeout_ms);
int net_listen(const char *host, int port, int backlog);
/* Return 1 for a frame, 0 for clean EOF, -1 with errno on failure. */
int net_recv_frame(int fd, char *buffer, size_t capacity, int timeout_ms, int robust);
int net_send_frame(int fd, const char *payload, int timeout_ms);

#endif
```

src/netsys.c

```
#define _POSIX_C_SOURCE 200809L
#include "netsys.h"

#include <arpa/inet.h>
#include <errno.h>
#include <fcntl.h>
#include <limits.h>
#include <netinet/in.h>
#include <poll.h>
#include <stdarg.h>
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <sys/socket.h>
#include <time.h>
#include <unistd.h>

static const char *log_role = "unknown";
static int runtime_debug;

void net_set_log(const char *role, int debug_enabled) {
    log_role = role;
    runtime_debug = debug_enabled;
}

int net_debug_enabled(void) { return runtime_debug; }

void net_log(const char *level, const char *format, ...) {
    char line[2048];
    struct timespec now;
    struct tm stamp;
```

```
    char time_text[32];
    clock_gettime(CLOCK_REALTIME, &now);
    localtime_r(&now.tv_sec, &stamp);
    strftime(time_text, sizeof(time_text), "%Y-%m-%dT%H:%M:%S", &stamp);
    int used = snprintf(line, sizeof(line), "%s.%03ld %s pid=%ld role=%s ",
                        time_text, now.tv_nsec / 1000000L, level, (long)getpid(), log_role);
    if (used < 0 || (size_t)used >= sizeof(line)) return;
    va_list args;
    va_start(args, format);
    int extra = vsnprintf(line + used, sizeof(line) - (size_t)used, format, args);
    va_end(args);
    if (extra < 0) return;
    size_t size = (size_t)used + (size_t)extra;
    if (size > sizeof(line) - 2) size = sizeof(line) - 2;
    line[size++] = '\n';
    /* One short write keeps individual process log records together. */
    ssize_t ignored = write(STDERR_FILENO, line, size);
    (void)ignored;
}

int64_t net_now_ms(void) {
    struct timespec now;
    if (clock_gettime(CLOCK_MONOTONIC, &now) < 0) return -1;
    return (int64_t)now.tv_sec * 1000 + now.tv_nsec / 1000000;
}

int net_parse_long(const char *text, long minimum, long maximum, long *value) {
    char *end;
    if (!text || !*text) return -1;
    errno = 0;
    long result = strtol(text, &end, 10);
    if (errno || *end || result < minimum || result > maximum) return -1;
    *value = result;
    return 0;
}

int net_nonblocking(int fd) {
    int flags = fcntl(fd, F_GETFL, 0);
    return flags < 0 ? -1 : fcntl(fd, F_SETFL, flags | O_NONBLOCK);
}

static int wait_ready(int fd, short events, int64_t deadline) {
    for (;;) {
        int timeout = -1;
        if (deadline >= 0) {
            int64_t remaining = deadline - net_now_ms();
            if (remaining <= 0) { errno = ETIMEDOUT; return -1; }
            timeout = remaining > INT_MAX ? INT_MAX : (int)remaining;
        }
        struct pollfd item = { .fd = fd, .events = events, .revents = 0 };
        int result = poll(&item, 1, timeout);
        if (result > 0) {
            if (item.revents & POLLNVAL) { errno = EBADF; return -1; }
            /* recv/send reports the precise error for HUP and ERR. */
            return 0;
        }
        if (result == 0) { errno = ETIMEDOUT; return -1; }
        if (errno != EINTR) return -1;
    }
}

static int read_bytes(int fd, void *buffer, size_t length, int64_t deadline,
                      int robust, int allow_clean_eof) {
    size_t received = 0;
    while (received < length) {
        if (wait_ready(fd, POLLIN, deadline) < 0) return -1;
        ssize_t count = recv(fd, (char *)buffer + received, length - received, 0);
        if (count == 0) {
            if (received == 0 && allow_clean_eof) return 0;
            errno = ECONNRESET;
            return -1;
        }
        if (count < 0) {
            if (errno == EINTR || errno == EAGAIN || errno == EWOULDBLOCK) continue;
            return -1;
        }
        received += (size_t)count;
        if (!robust && received < length) { errno = EPROTO; return -1; }
    }
    return 1;
}

int net_recv_frame(int fd, char *buffer, size_t capacity, int timeout_ms, int robust) {
```

```
    uint32_t wire_length;
    int64_t deadline = timeout_ms < 0 ? -1 : net_now_ms() + timeout_ms;
    int result = read_bytes(fd, &wire_length, sizeof(wire_length), deadline, robust, 1);
    if (result <= 0) return result;
    uint32_t length = ntohl(wire_length);
    if (length == 0 || length > NET_MAX_FRAME || (size_t)length >= capacity) {
        errno = EMSGSIZE;
        return -1;
    }
    result = read_bytes(fd, buffer, length, deadline, robust, 0);
    if (result < 0) return -1;
    if (memchr(buffer, '\0', length)) { errno = EPROTO; return -1; }
    buffer[length] = '\0';
    NET_DEBUG("FRAME_RX bytes=%u", length);
    return 1;
}

int net_send_frame(int fd, const char *payload, int timeout_ms) {
    size_t length = strlen(payload);
    if (length == 0 || length > NET_MAX_FRAME) { errno = EMSGSIZE; return -1; }
    unsigned char frame[NET_MAX_FRAME + 4];
    uint32_t wire_length = htonl((uint32_t)length);
    memcpy(frame, &wire_length, 4);
    memcpy(frame + 4, payload, length);
    size_t sent = 0;
    int64_t deadline = timeout_ms < 0 ? -1 : net_now_ms() + timeout_ms;
    while (sent < length + 4) {
        if (wait_ready(fd, POLLOUT, deadline) < 0) return -1;
        ssize_t count = send(fd, frame + sent, length + 4 - sent, MSG_NOSIGNAL);
        if (count < 0) {
            if (errno == EINTR || errno == EAGAIN || errno == EWOULDBLOCK) continue;
            return -1;
        }
        if (count == 0) { errno = EPIPE; return -1; }
        sent += (size_t)count;
    }
    NET_DEBUG("FRAME_TX bytes=%zu", length);
    return 0;
}

static int make_address(const char *host, int port, struct sockaddr_in *address) {
    memset(address, 0, sizeof(*address));
    address->sin_family = AF_INET;
    address->sin_port = htons((uint16_t)port);
    if (inet_pton(AF_INET, host, &address->sin_addr) != 1) { errno = EINVAL; return -1; }
    return 0;
}

int net_connect(const char *host, int port, int timeout_ms) {
    struct sockaddr_in address;
    if (make_address(host, port, &address) < 0) return -1;
    int fd = socket(AF_INET, SOCK_STREAM, 0);
    if (fd < 0) return -1;
    if (net_nonblocking(fd) < 0) goto failure;
    if (connect(fd, (struct sockaddr *)&address, sizeof(address)) == 0) return fd;
    if (errno != EINPROGRESS) goto failure;
    if (wait_ready(fd, POLLOUT, net_now_ms() + timeout_ms) < 0) goto failure;
    int socket_error = 0;
    socklen_t size = sizeof(socket_error);
    if (getsockopt(fd, SOL_SOCKET, SO_ERROR, &socket_error, &size) < 0) goto failure;
    if (socket_error) { errno = socket_error; goto failure; }
    return fd;
failure: {
        int saved = errno;
        close(fd);
        errno = saved;
        return -1;
    }
}

int net_listen(const char *host, int port, int backlog) {
    struct sockaddr_in address;
    if (make_address(host, port, &address) < 0) return -1;
    int fd = socket(AF_INET, SOCK_STREAM, 0);
    if (fd < 0) return -1;
    int enabled = 1;
    if (setsockopt(fd, SOL_SOCKET, SO_REUSEADDR, &enabled, sizeof(enabled)) < 0 ||
        net_nonblocking(fd) < 0 ||
        bind(fd, (struct sockaddr *)&address, sizeof(address)) < 0 ||
        listen(fd, backlog) < 0) {
        int saved = errno;
        close(fd);
        errno = saved;
```

```
198         return -1;
199     }
200     return fd;
201 }
```

src/server.c

```
  1 #define _POSIX_C_SOURCE 200809L
  2 #include "netsys.h"
  3
  4 #include <arpa/inet.h>
  5 #include <errno.h>
  6 #include <getopt.h>
  7 #include <poll.h>
  8 #include <signal.h>
  9 #include <stdio.h>
 10 #include <stdlib.h>
 11 #include <string.h>
 12 #include <sys/socket.h>
 13 #include <sys/sysinfo.h>
 14 #include <sys/types.h>
 15 #include <sys/utsname.h>
 16 #include <sys/wait.h>
 17 #include <time.h>
 18 #include <unistd.h>
 19
 20 #define CHILD_LIMIT 1024
 21 static volatile sig_atomic_t stopping;
 22 static void stop_handler(int number) { (void)number; stopping = 1; }
 23 static void child_handler(int number) { (void)number; }
 24
 25 static int set_handler(int number, void (*handler)(int)) {
 26     struct sigaction action;
 27     memset(&action, 0, sizeof(action));
 28     action.sa_handler = handler;
 29     sigemptyset(&action.sa_mask);
 30     return sigaction(number, &action, NULL);
 31 }
 32
 33 static void usage(FILE *stream, const char *program) {
 34     fprintf(stream, "Usage: %s [--host IPv4] [--port N] [--timeout-ms N] [--max-clients N] [--debug]
    [--unsafe]\n"
 35                     "--unsafe is a teaching mode: single recv, no idle deadline, no child reaping.\n",
    program);
 36 }
 37
 38 static void system_info(char *response, size_t size) {
 39     struct utsname name;
 40     struct sysinfo info;
 41     char hostname[256];
 42     if (uname(&name) < 0 || sysinfo(&info) < 0 || gethostname(hostname, sizeof(hostname)) < 0) {
 43         snprintf(response, size, "ERR SYSINFO %s", strerror(errno));
 44         return;
 45     }
 46     hostname[sizeof(hostname) - 1] = '\0';
 47     snprintf(response, size,
 48              "OK SYSINFO\nhostname=%s\nsysname=%s\nrelease=%s\nmachine=%s\npid=%ld\n"
 49              "uptime_seconds=%ld\nram_total_bytes=%llu\nram_free_bytes=%llu",
 50              hostname, name.sysname, name.release, name.machine, (long)getpid(),
 51              info.uptime, (unsigned long long)info.totalram * info.mem_unit,
 52              (unsigned long long)info.freeram * info.mem_unit);
 53 }
 54
 55 static int handle_command(const char *request, char *response, size_t size) {
 56     if (strcmp(request, "PING") == 0) {
 57         snprintf(response, size, "OK PONG pid=%ld", (long)getpid());
 58     } else if (strcmp(request, "SYSINFO") == 0) {
 59         system_info(response, size);
 60     } else if (strncmp(request, "ECHO ", 5) == 0) {
 61         if (strlen(request + 5) > NET_MAX_FRAME - 3) snprintf(response, size, "ERR ECHO_TOO_LONG");
 62         else snprintf(response, size, "OK %s", request + 5);
 63     } else if (strncmp(request, "SLEEP ", 6) == 0) {
 64         long milliseconds;
 65         if (net_parse_long(request + 6, 0, 10000, &milliseconds) < 0) {
 66             snprintf(response, size, "ERR BAD_ARGUMENT SLEEP accepts 0..10000 milliseconds");
 67         } else {
 68             struct timespec delay = { .tv_sec = milliseconds / 1000, .tv_nsec = (milliseconds % 1000) *
    1000000L };
 69             while (nanosleep(&delay, &delay) < 0 && errno == EINTR) { }
 70             snprintf(response, size, "OK SLEPT %ld pid=%ld", milliseconds, (long)getpid());
```

```
        }
    } else if (strcmp(request, "QUIT") == 0) {
        snprintf(response, size, "OK BYE");
        return 1;
    } else {
        snprintf(response, size, "ERR UNKNOWN_COMMAND");
    }
    return 0;
}

static void serve_client(int fd, int timeout, int robust, int debug) {
    net_set_log("worker", debug);
    char request[NET_MAX_FRAME + 1], response[NET_MAX_FRAME + 1];
    for (;;) {
        int result = net_recv_frame(fd, request, sizeof(request), robust ? timeout : -1, robust);
        if (result == 0) { NET_DEBUG("PEER_EOF"); break; }
        if (result < 0) {
            int saved = errno;
            net_log("WARN", "FRAME_READ_ERROR errno=%d %s", saved, strerror(saved));
            const char *error = saved == ETIMEDOUT ? "ERR TIMEOUT" :
                                saved == EMSGSIZE ? "ERR BAD_LENGTH" : "ERR BAD_FRAME";
            (void)net_send_frame(fd, error, 300);
            break;
        }
        NET_DEBUG("COMMAND bytes=%zu", strlen(request));
        int quit = handle_command(request, response, sizeof(response));
        if (net_send_frame(fd, response, timeout) < 0) {
            net_log("WARN", "FRAME_WRITE_ERROR errno=%d %s", errno, strerror(errno));
            break;
        }
        if (quit) break;
    }
    close(fd);
    net_log("INFO", "CHILD_DONE");
}

static void remove_child(pid_t *children, size_t *count, pid_t pid) {
    for (size_t index = 0; index < *count; ++index) {
        if (children[index] == pid) {
            children[index] = children[--(*count)];
            return;
        }
    }
}

static void reap_children(pid_t *children, size_t *count) {
    int status;
    pid_t pid;
    while ((pid = waitpid(-1, &status, WNOHANG)) > 0) {
        remove_child(children, count, pid);
        net_log("INFO", "REAP child=%ld active=%zu status=%d", (long)pid, *count, status);
    }
}

int main(int argc, char **argv) {
    const char *host = "127.0.0.1";
    int port = NET_DEFAULT_PORT, timeout = 2000, max_clients = 64, debug = 0, robust = 1;
    static const struct option options[] = {
        {"host", required_argument, NULL, 'H'}, {"port", required_argument, NULL, 'p'},
        {"timeout-ms", required_argument, NULL, 't'}, {"max-clients", required_argument, NULL, 'm'},
        {"debug", no_argument, NULL, 'd'}, {"unsafe", no_argument, NULL, 'u'},
        {"help", no_argument, NULL, 'h'}, {NULL, 0, NULL, 0}
    };
    int option;
    long value;
    while ((option = getopt_long(argc, argv, "H:p:t:m:duh", options, NULL)) != -1) {
        switch (option) {
        case 'H': host = optarg; break;
        case 'p': if (net_parse_long(optarg, 1, 65535, &value) < 0) goto invalid; port = (int)value; break;
        case 't': if (net_parse_long(optarg, 1, 600000, &value) < 0) goto invalid; timeout = (int)value;
    break;
        case 'm': if (net_parse_long(optarg, 1, CHILD_LIMIT, &value) < 0) goto invalid; max_clients =
    (int)value; break;
        case 'd': debug = 1; break;
        case 'u': robust = 0; break;
        case 'h': usage(stdout, argv[0]); return 0;
        default: goto invalid;
        }
    }
    if (optind != argc) goto invalid;
    net_set_log("server", debug);
    if (set_handler(SIGTERM, stop_handler) < 0 || set_handler(SIGINT, stop_handler) < 0 ||
        set_handler(SIGCHLD, child_handler) < 0 || set_handler(SIGPIPE, SIG_IGN) < 0) {
```

```
152         net_log("ERROR", "SIGNAL_SETUP_FAILED %s", strerror(errno)); return 1;
153     }
154     int listener = net_listen(host, port, 128);
155     if (listener < 0) { net_log("ERROR", "LISTEN_FAILED errno=%d %s", errno, strerror(errno)); return 1; }
156     pid_t children[CHILD_LIMIT];
157     size_t count = 0;
158     net_log("INFO", "READY host=%s port=%d robust=%d timeout_ms=%d max_clients=%d compiled_debug=%d",
159             host, port, robust, timeout, max_clients, ENABLE_DEBUG);
160     if (!robust) net_log("WARN", "UNSAFE_TEACHING_MODE partial_read_recovery=off idle_deadline=off
    child_reaping=off");
161     NET_DEBUG("DEBUG_BUILD runtime_debug=on");
162     while (!stopping) {
163         if (robust) reap_children(children, &count);
164         struct pollfd item = { .fd = listener, .events = POLLIN, .revents = 0 };
165         int ready = poll(&item, 1, 100);
166         if (ready < 0) {
167             if (errno == EINTR) continue;
168             net_log("ERROR", "POLL_FAILED %s", strerror(errno));
169             break;
170         }
171         if (ready == 0) continue;
172         struct sockaddr_in peer;
173         socklen_t peer_size = sizeof(peer);
174         int fd = accept(listener, (struct sockaddr *)&peer, &peer_size);
175         if (fd < 0) {
176             if (errno == EINTR || errno == EAGAIN || errno == EWOULDBLOCK || errno == ECONNABORTED)
    continue;
177             net_log("WARN", "ACCEPT_FAILED errno=%d %s", errno, strerror(errno));
178             struct timespec pause = { .tv_sec = 0, .tv_nsec = 100000000L };
179             nanosleep(&pause, NULL);
180             continue;
181         }
182         if (net_nonblocking(fd) < 0) { close(fd); continue; }
183         if (robust) reap_children(children, &count);
184         if (count >= (size_t)max_clients) {
185             net_log("WARN", "CLIENT_LIMIT active=%zu max=%d", count, max_clients);
186             (void)net_send_frame(fd, "ERR SERVER_BUSY", 100);
187             close(fd);
188             continue;
189         }
190         /* A worker must install default termination handlers before an early
191            parent shutdown can send SIGTERM. Otherwise it could inherit the
192            parent's flag-only handler and keep serving while waitpid blocks. */
193         sigset_t termination_signals, old_mask;
194         sigemptyset(&termination_signals);
195         sigaddset(&termination_signals, SIGINT);
196         sigaddset(&termination_signals, SIGTERM);
197         if (sigprocmask(SIG_BLOCK, &termination_signals, &old_mask) < 0) {
198             net_log("ERROR", "SIGNAL_MASK_FAILED %s", strerror(errno));
199             close(fd);
200             continue;
201         }
202         pid_t child = fork();
203         if (child < 0) {
204             int saved = errno;
205             (void)sigprocmask(SIG_SETMASK, &old_mask, NULL);
206             errno = saved;
207             net_log("ERROR", "FORK_FAILED errno=%d %s", errno, strerror(errno));
208             (void)net_send_frame(fd, "ERR SERVER_BUSY", 100);
209             close(fd);
210             continue;
211         }
212         if (child == 0) {
213             close(listener);
214             set_handler(SIGTERM, SIG_DFL);
215             set_handler(SIGINT, SIG_DFL);
216             set_handler(SIGCHLD, SIG_DFL);
217             (void)sigprocmask(SIG_SETMASK, &old_mask, NULL);
218             serve_client(fd, timeout, robust, debug);
219             _exit(0);
220         }
221         close(fd);
222         children[count++] = child;
223         (void)sigprocmask(SIG_SETMASK, &old_mask, NULL);
224         char peer_text[INET_ADDRSTRLEN] = "?";
225         inet_ntop(AF_INET, &peer.sin_addr, peer_text, sizeof(peer_text));
226         net_log("INFO", "FORK child=%ld active=%zu peer=%s:%u", (long)child, count,
227                 peer_text, (unsigned)ntohs(peer.sin_port));
228     }
229     close(listener);
230     /* Shutdown deliberately reaps in both modes so the comparison leaves no debris. */
231     for (size_t index = 0; index < count; ++index) (void)kill(children[index], SIGTERM);
232     while (count > 0) {
```

```
        int status;
        pid_t child = waitpid(-1, &status, 0);
        if (child > 0) remove_child(children, &count, child);
        else if (errno != EINTR) break;
    }
    net_log("INFO", "STOPPED active=%zu", count);
    return 0;
invalid:
    usage(stderr, argv[0]);
    return 2;
}
```

src/client.c

```
#define _POSIX_C_SOURCE 200809L
#include "netsys.h"

#include <errno.h>
#include <getopt.h>
#include <signal.h>
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <unistd.h>

static void usage(FILE *stream, const char *program) {
    fprintf(stream, "Usage: %s [--host IPv4] [--port N] [--timeout-ms N] [--debug] COMMAND [ARG ...]\n"
                    "Commands: SYSINFO | PING | ECHO text | SLEEP milliseconds | QUIT\n", program);
}

int main(int argc, char **argv) {
    const char *host = "127.0.0.1";
    int port = NET_DEFAULT_PORT, timeout = 5000, debug = 0;
    static const struct option options[] = {
        {"host", required_argument, NULL, 'H'}, {"port", required_argument, NULL, 'p'},
        {"timeout-ms", required_argument, NULL, 't'}, {"debug", no_argument, NULL, 'd'},
        {"help", no_argument, NULL, 'h'}, {NULL, 0, NULL, 0}
    };
    int option;
    long value;
    while ((option = getopt_long(argc, argv, "+H:p:t:dh", options, NULL)) != -1) {
        switch (option) {
        case 'H': host = optarg; break;
        case 'p': if (net_parse_long(optarg, 1, 65535, &value) < 0) goto invalid; port = (int)value; break;
        case 't': if (net_parse_long(optarg, 1, 600000, &value) < 0) goto invalid; timeout = (int)value;
break;
        case 'd': debug = 1; break;
        case 'h': usage(stdout, argv[0]); return 0;
        default: goto invalid;
        }
    }
    if (optind >= argc) goto invalid;
    net_set_log("client", debug);
    signal(SIGPIPE, SIG_IGN);
    char request[NET_MAX_FRAME + 1] = "";
    size_t used = 0;
    for (int index = optind; index < argc; ++index) {
        size_t length = strlen(argv[index]);
        size_t separator = index == optind ? 0 : 1;
        if (used + separator + length > NET_MAX_FRAME) {
            net_log("ERROR", "REQUEST_TOO_LARGE max=%u", NET_MAX_FRAME);
            return 2;
        }
        if (separator) request[used++] = ' ';
        memcpy(request + used, argv[index], length);
        used += length;
        request[used] = '\0';
    }
    NET_DEBUG("CONNECT host=%s port=%d timeout_ms=%d", host, port, timeout);
    int fd = net_connect(host, port, timeout);
    if (fd < 0) { net_log("ERROR", "CONNECT_FAILED errno=%d %s", errno, strerror(errno)); return 1; }
    if (net_send_frame(fd, request, timeout) < 0) {
        net_log("ERROR", "SEND_FAILED errno=%d %s", errno, strerror(errno)); close(fd); return 1;
    }
    char response[NET_MAX_FRAME + 1];
    int result = net_recv_frame(fd, response, sizeof(response), timeout, 1);
    if (result <= 0) {
        if (result == 0) net_log("ERROR", "SERVER_CLOSED_WITHOUT_RESPONSE");
        else net_log("ERROR", "RECEIVE_FAILED errno=%d %s", errno, strerror(errno));
        close(fd);
```

```
66          return 1;
67      }
68      puts(response);
69      close(fd);
70      return strncmp(response, "OK", 2) == 0 ? 0 : 1;
71  invalid:
72      usage(stderr, argv[0]);
73      return 2;
74  }
```

Makefile

```
 1  CC ?= gcc
 2  DEBUG ?= 1
 3  ifeq ($(DEBUG),1)
 4  MODE := debug
 5  OPT := -O0 -g3
 6  else ifeq ($(DEBUG),0)
 7  MODE := release
 8  OPT := -O2
 9  else
10  $(error DEBUG must be 0 or 1)
11  endif
12
13  BUILD := build/$(MODE)
14  CPPFLAGS += -Iinclude -DENABLE_DEBUG=$(DEBUG)
15  CFLAGS += -std=c11 -Wall -Wextra -Wpedantic -Werror $(OPT)
16  LDFLAGS +=
17
18  .PHONY: all clean test debug release
19  all: $(BUILD)/server $(BUILD)/client
20
21  $(BUILD):
22     mkdir -p $@
23
24  $(BUILD)/netsys.o: src/netsys.c include/netsys.h | $(BUILD)
25     $(CC) $(CPPFLAGS) $(CFLAGS) -fPIC -c $< -o $@
26
27  $(BUILD)/libnetsys.so: $(BUILD)/netsys.o
28     $(CC) -shared $(LDFLAGS) -Wl,-soname,libnetsys.so $< -o $@
29
30  $(BUILD)/server: src/server.c include/netsys.h $(BUILD)/libnetsys.so
31     $(CC) $(CPPFLAGS) $(CFLAGS) $< -L$(BUILD) -lnetsys -Wl,-rpath,'$$ORIGIN' $(LDFLAGS) -o $@
32
33  $(BUILD)/client: src/client.c include/netsys.h $(BUILD)/libnetsys.so
34     $(CC) $(CPPFLAGS) $(CFLAGS) $< -L$(BUILD) -lnetsys -Wl,-rpath,'$$ORIGIN' $(LDFLAGS) -o $@
35
36  debug:
37     $(MAKE) DEBUG=1 all
38
39  release:
40     $(MAKE) DEBUG=0 all
41
42  test: all
43     python3 tests/run_tests.py
44
45  clean:
46     rm -rf build
```